Pointers

In C++

A pointer is a variable whose value is the address of another variable. Like any variable or constant, you must declare a pointer before you can work with it.

Declaring and Initializing Pointers:

The declaration of the pointer is based on the data type of the variable it points to.

E.g.

*data-type *pointer-variable;*

At any point of time a pointer variable can point to only one data type.

int *ptr;

Here ptr contains the memory location of any integer variable.

e.g. int *ptr, a; //Declaration

ptr=&a; //Initialization

Pointers in C++

1. Null Pointers	C++ supports null pointer, which is a constant with a value of zero defined in several standard libraries.
2. Pointer Arithmetic	There are four arithmetic operators that can be used on pointers: ++, --, +, -
3. Pointers vs Arrays	There is a close relationship between pointers and arrays.
4. Array of Pointers	You can define arrays to hold a number of pointers.
5. Pointer to Pointer	C++ allows you to have pointer on a pointer and so on.

Manipulation of Pointers

There are 2 important parts when it comes to using a pointer:
1) Referencing
2) Dereferencing

To be Noted while using pointers:

1. Dereferencing an uninitialized pointer can cause a crash
2. Dereferencing a different data type's pointer can cause a crash
3. Trying to Dereference a pointer to a variable that is out of scope can also cause a crash

Referencing a Pointer

Referencing means taking an Address of an existing variable and storing it in a pointer.
Here we use the Ampersand (&) symbol.
Take a look at the following code:

```
#include <iostream>
using namespace std;
int main()
{
int a;
int* b;
a = 123;
b = &a; }
```

Here b references a.
So 123 gets stored in a.
b that is of a pointer type stores the address of a.

Dereferencing a Pointer in C++

Dereferencing a pointer means taking the address stored in a pointer and finding the value the address points to.
We use the Asterix (*) symbol here.
Take a look at the code below:

```
#include <iostream>
using namespace std;
int main()
{
int a,c;
int* b;
a = 123;
b = &a;
c = *b;
}
```

Till the “b = &a” line, everything is exactly the same as the code in referencing a pointer
But the “c = *b ” is an add on.
Here c gets the value of b, which is pointing to.a
It gets the address from *b, finds the address, gets the value stored in there (i.e. 123).

So if we print c using ” cout<<c; “,
we should get 123 as output

Pointer Expression and Pointer Arithmetic

Normally following arithmetic operations are performed.

- A pointer can be incremented(++) or decremented(--).
- Any Integer can be added to or decremented from a pointer.
- One Pointer can be subtracted from other.

e.g. int a[6];

int *aptr;

aptr=&a[0];

Here aptr refers to the base address of array a.

By Incrementing aptr (i.e. aptr++)we can move to the next location in array.

Similarly by decrementing aptr (i.e. aptr--)we can move to the previous location in array.

Imp: Here Pointer arithmetic is applicable as the elements are at the neighbouring locations.

Array of Pointers

An array of Pointer is an array of addresses i.e all the elements of the Pointer array points to an item of the data array. An array of Pointer is declared as

int *abc[10];

It is an array of ten pointers ie. Addresses all of them points to an integer. Before initializing they point to some unknown values in the memory space.

We can declare pointers to arrays as

int *ptr;

ptr = number[0]; OR ptr = number;

Example:

```
#include <iostream>
 using namespace std;
const int MAX = 4;

int main () {
const char *names[MAX] = { "Zara Ali", "Hina Ali", "Nuha Ali", "Sara Ali" };

  for (int i = 0; i < MAX; i++) {
      cout << "Value of names[" << i << "] = ";
      cout << (names + i) << endl;
   }
   return 0;
}
```

Output:

```
Value of names[0] = 0x7ffd256683c0
Value of names[1] = 0x7ffd256683c8
Value of names[2] = 0x7ffd256683d0
Value of names[3] = 0x7ffd256683d8
```

Pointers & Strings

A pointer variable can access a string by referring to its first character. We can assign the values to the arrays in two ways:

char num[]=“one”;

here an array of four characters is declared ‘o’, ‘n’, ‘e’ and ‘\0’

const char *numptr=“one”;

here a pointer variable is generated which points to the first character ‘o’ of string.

Example:

```
string food = "Pizza";  // A food variable of type string
string* ptr = &food;    // A pointer variable, with the name ptr, that stores the address of food

// Output the value of food (Pizza)
cout << food << "\n";

// Output the memory address of food (0x6dfed4)
cout << &food << "\n";

// Output the memory address of food with the pointer (0x6dfed4)
cout << ptr << "\n";
```

Here, a pointer variable is created with the name ptr, that points to a string variable, by using the asterisk sign * (string* ptr). Note that the type of the pointer has to match the type of the variable you're working with.

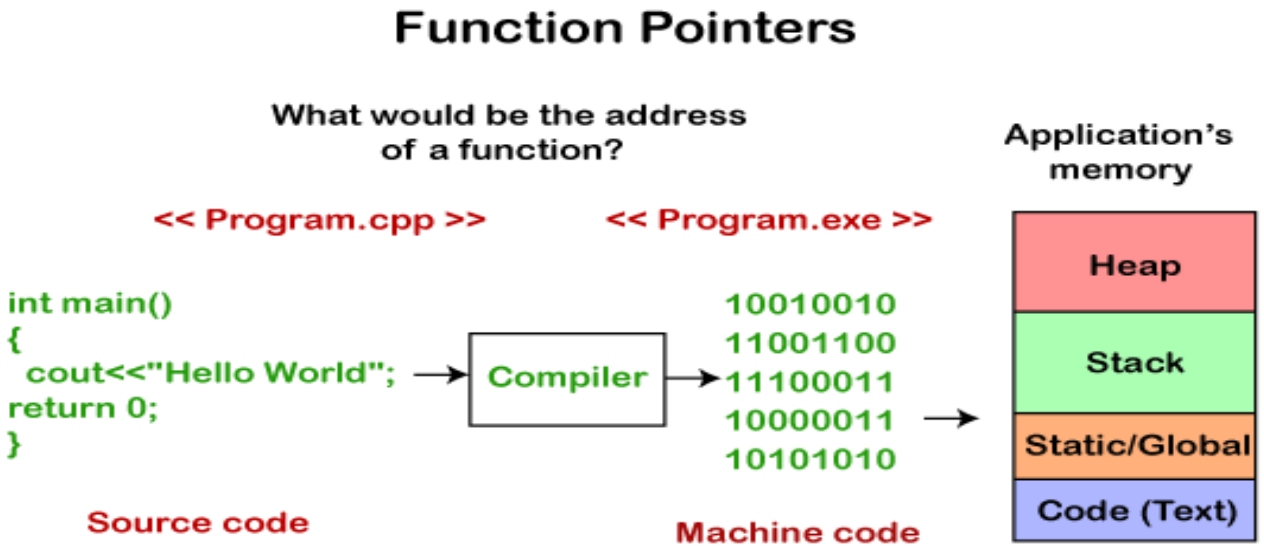
Used the & operator to store the memory address of the variable called food, and assign it to the pointer.

Now, ptr holds the value of food's memory address.

Pointers to Functions

The Pointer to function is also known as *callback function.*

All the functions and machine code instructions are data. This data is a bunch of bytes, and all these bytes have some address in RAM. The function pointer contains RAM address of the first instruction of a function.



Pointers to Functions

Pointers to functions are used for :

- Referring to a function
- Function can be selected dynamically at run time.
- Function can be passed as an argument to another function.
- There are two types of Function Pointers

(i) Points to Static Members Functions.

(ii) Points to non static member functions.(It requires hidden argument)

Pointer to Function is Declared as:

```
data_type(*function_name)();
```

Example:

```
#include <iostream>
using namespace std;
int add(int a , int b)
{  return a+b;  }
int main()  {
 int (*funcptr)(int,int);  // function pointer declaration
 funcptr=add; // funcptr is pointing to the add function
 int sum=funcptr(5,5);
 std::cout << "value of sum is :" <<sum<< std::endl;
  return 0;  }
```

Output:

value of sum is :10

In the above program, we declare the function pointer, i.e., int (*funcptr)(int,int) and then we store the address of add() function in funcptr. This implies that funcptr contains the address of add() function. Now, we can call the add() function by using funcptr. The statement funcptr(5,5) calls the add() function, and the result of add() function gets stored in sum variable.

Pointers to objects

A Pointer can also point to the object created by a class.
For example:

```
Class item
{
        int code;
        float price;
 public:
    void getdata(int a , float b)
          {
            code=a;
            price=b;
          }
   void show()
          {
            cout<<“Code :  ”<< code<<“\n”;
             cout<<“Price : ”<< price<<“\n”;
          }
};
```

```
item x;
item *ptr=&x;       // Now the member functions can be
      pointed in two ways.
                    x.getdata(100,75.5);
                    x.show();
                          OR
                    ptr->getdata(100, 75.5);
                    ptr->show();                    OR
                    (*ptr).show();
```

Pointers to objects

The same declaration can also be done as:

item *ptr=**new** item;

Here new operator allocates sufficient memory to ptr.

Imp: If a class has a constructor with parameters and does not include any empty constructor. Then we must pass the parameters when we create any object.

this Pointer

“this” represents an object that invokes a member function. this is a pointer that points
to the object for which *this* function was called.

The pointer this acts as an implicit argument to all the member functions.

Following are the situations where ‘this’ pointer:

1) When local variable’s name is same as member’s name

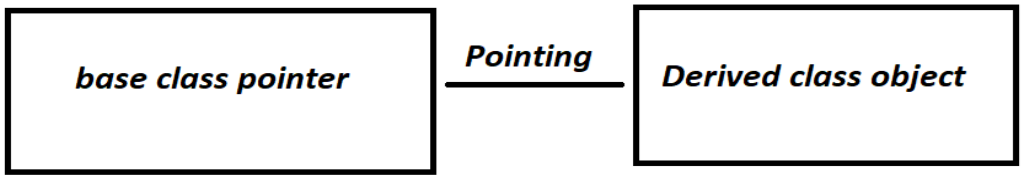
2) To return reference to the calling object

```
#include<iostream>
using namespace std;
/* local variable is same as a member's name */
class Test{
private:
int x;
public:
void setX (int x)
{ // The 'this' pointer is used to retrieve the object's x
                    // hidden by the local variable 'x'
                    this->x = x; }
void print() { cout << "x = " << x << endl; }};
int main()
{
Test obj;
int x = 20;
obj.setX(x);
obj.print();
return 0;
}
```

Output:

Pointers to Derived Classes

We can declare the pointers to the base class as well as the pointers to the derived classes. Pointers to the base classes are type-compatible to the objects of the derived classes. Hence a single pointer can be made to point to the different classes.



Used to point the derived class object and pointer can still use aspects of bsse class

E.g. Here B is a base class, D is Derived class.

```
B *cptr;
B b;
D d;
cptr = &b;
```

we can also make the cptr to point to object d also.

cptr = &d;

As d is an object of the class D derived from the class B.

we can make in the above example cptr to point to the object d as follows
cptr = &d; // cptr points to object d

This is perfectly valid with c++ language because d is an object derived from the class B.